交通灯控制器：全部 Verilog 代码

层次化原理图使用的三个器件：计时器、状态机与灯光译码器

1. 一秒计时器（timer_1s.v）

输入 50 MHz 时钟，输出每秒一次的 tick_1s 脉冲。

```
module timer_1s (
    input wire clk,
    input wire rst_n,
    output reg tick_1s
);

    parameter COUNT_MAX = 26'd49999999;

    reg [25:0] count;

    always @(posedge clk or negedge rst_n)
    begin
        if (!rst_n)
        begin
            count <= 26'd0;
            tick_1s <= 1'b0;
        end
        else if (count == COUNT_MAX)
        begin
            count <= 26'd0;
            tick_1s <= 1'b1;
        end
        else
        begin
            count <= count + 1'b1;
            tick_1s <= 1'b0;
        end
    end

endmodule
```

2. 交通灯状态机（traffic_fsm.v）

按 30 秒、5 秒、20 秒、5 秒的顺序切换交通状态。

```
`timescale 1ns/1ps

// 器件一：保存交通灯当前阶段并完成 30/5/20/5 秒倒计时。
module traffic_fsm (
    input  wire clk,
    input  wire rst_n,
    input  wire tick_1s,
    output wire state_0,
    output wire state_1
);
    // 状态编码：parameter 是教材中定义符号常量的写法。
    parameter MAIN_GREEN  = 2'd0;
    parameter MAIN_YELLOW = 2'd1;
    parameter SIDE_GREEN  = 2'd2;
    parameter SIDE_YELLOW = 2'd3;

    reg [1:0] state;
    reg [5:0] seconds_left;

    assign state_0 = state[0];
    assign state_1 = state[1];

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state        <= MAIN_GREEN;
            seconds_left <= 6'd30;
        end else if (tick_1s) begin
            if (seconds_left > 6'd1)
                seconds_left <= seconds_left - 6'd1;
            else begin
                case (state)
                    MAIN_GREEN:  begin state <= MAIN_YELLOW; seconds_left <= 6'd5;  end
                    MAIN_YELLOW: begin state <= SIDE_GREEN;  seconds_left <= 6'd20; end
                    SIDE_GREEN:  begin state <= SIDE_YELLOW; seconds_left <= 6'd5;  end
                    default:     begin state <= MAIN_GREEN;  seconds_left <= 6'd30; end
                endcase
            end
        end
    end
endmodule
```

3. 灯光译码器（traffic_decoder.v）

将两位状态码转换为主路、支路的六个灯输出。

```
`timescale 1ns/1ps

// 器件二：把两位状态码译成主路和支路的六个灯输出。
module traffic_decoder (
    input  wire state_0,
    input  wire state_1,
    output reg  main_red,
    output reg  main_yellow,
    output reg  main_green,
    output reg  side_red,
    output reg  side_yellow,
    output reg  side_green
);
    always @(state_0 or state_1)
    begin
        main_red = 1'b0;
        main_yellow = 1'b0;
        main_green = 1'b0;

        side_red = 1'b0;
        side_yellow = 1'b0;
        side_green = 1'b0;

        case ({state_1, state_0})
            2'b00:
            begin
                main_green = 1'b1;
                side_red = 1'b1;
            end

            2'b01:
            begin
                main_yellow = 1'b1;
                side_red = 1'b1;
            end

            2'b10:
            begin
                main_red = 1'b1;
                side_green = 1'b1;
            end

            default:
            begin
                main_red = 1'b1;
                side_yellow = 1'b1;
            end
        endcase
    end
endmodule
```